Deliverable 2 (D2)

Campus (NC/OC)	
Project ID	
Project Title	
Team Lead Roll Number	
Supervisor	

Capstone Project Deliverable – 2

AI-Based Real-Time Vehicle Smoke Detection and Reporting System

Department/Campus	Roll Number	Name	Member / Team Lead	Signature
IT-OC	BITF22M031	Maheen Fatima	Team Lead	
IT-OC	BITF22M026	Areeba Ahmad	Member	
IT-OC	BITF22M037	Rabia Sadiq	Member	
IT-OC	BITF22M043	Laiba Saleem	Member	

Supervisor Name	Department	Email	Primary / Secondary	Signatures
Dr. Muhammad Farooq	IT	mfarooq@pucit.edu.pk	Primary	

Department of Information Technology
Faculty of Computing & Information Technology
University of the Punjab

Table of Contents

Executive Summary 4
Design Details 4
Use Case Model 5
Domain Model / Entity Model 7
ER Model (Data Model) 9
System Behavioral Model (Sequence Diagram) 12
Page Design 13
CASE Tools 18
Appendix 20
A. Overview of CASE (Computer-Aided Software Engineering) Tools 20
B. CASE Tools for System Modeling 20
C. CASE Tools for AI Development & Validation 21
D. CASE Tools for Backend & API Testing 21
E. CASE Tools for Real-Time Communication & Deployment 22
F. CASE Tools for Repository & Version Control 22
G. CASE Tools for Project Management & Communication 22
Appendix Summary 23

Executive Summary

The rapid increase in vehicular emissions has become a major contributor to urban air pollution. Manual monitoring of high-emission vehicles is inefficient, inconsistent, and difficult to scale across large cities. Existing surveillance infrastructure is underutilized for environmental enforcement purposes.

This project presents an **AI-Based Real-Time Vehicle Smoke Detection and Reporting System** designed to assist environmental and traffic authorities. The system analyzes video feeds from existing traffic cameras to detect visible smoke emissions from vehicles using computer vision and deep learning techniques. Upon detection, the system captures high-quality visual evidence and generates structured violation records.

The solution is implemented as a **non-invasive middleware**, requiring no specialized hardware. While the current prototype processes recorded video files for validation, the system architecture is designed to support real-time video streams in future deployments. Role-Based Access Control (RBAC) is enforced to ensure data privacy and secure access to sensitive information.

This deliverable focuses on the **detailed system design**, including architecture, component interaction, data models, and user roles. The work plan includes finalizing design artifacts, implementing core modules, testing detection accuracy, and preparing the system for controlled deployment and evaluation.

The proposed design ensures scalability, maintainability, and clarity of system components, enabling smooth transition from design to implementation phases. The documentation is prepared in alignment with Deliverable-01 and the approved project proposal.

Design Details

This section establishes the solution design using **Unified Modeling Language (UML)** at the **system (base) level**. The objective is to describe how the system behaves as a whole by identifying its major inputs, processing stages, and intended outputs.

At the system level, the proposed solution accepts **video input** from traffic surveillance sources, either as recorded files (current prototype) or live camera streams (proposed deployment). These inputs are processed through an AI-based inference pipeline that detects visible vehicular smoke emissions. Upon successful detection, the system produces structured outputs in the form of alerts, evidence records, and analytical reports.

The design emphasizes **modularity and abstraction**, where internal implementation details of AI models and storage mechanisms are encapsulated within subsystems. This allows the system to remain scalable, maintainable, and adaptable to real-time operation without altering the core design. UML diagrams are used to clearly define system interactions, user responsibilities, and data flow while avoiding unnecessary implementation-level complexity.

Use Case Model

A use case represents an interaction between an **actor** and the **system**, describing what the system does in response to a specific input. The use case model captures the functional behavior of the system from the user's perspective and defines the boundaries of system responsibilities.

The following design guidelines have been applied:

- A limited and appropriate set of **user-side actors** is selected.
- Each use case clearly represents an **action (input)** and a corresponding **system response (output)**.
- Use cases are kept **concise and controlled**, avoiding low-level operational details.
- Major system functionalities are grouped into **subsystem packages**.
- Proactive and reactive behaviors are explicitly identified.

Identified Actors

- **System Administrator**
- **Environmental / Traffic Officer**
- **Data Analyst**

Subsystem Packages

- User Management Subsystem
- Detection & Monitoring Subsystem
- Evidence & Reporting Subsystem
- Analytics & Visualization Subsystem

Proactive vs Reactive Use Cases

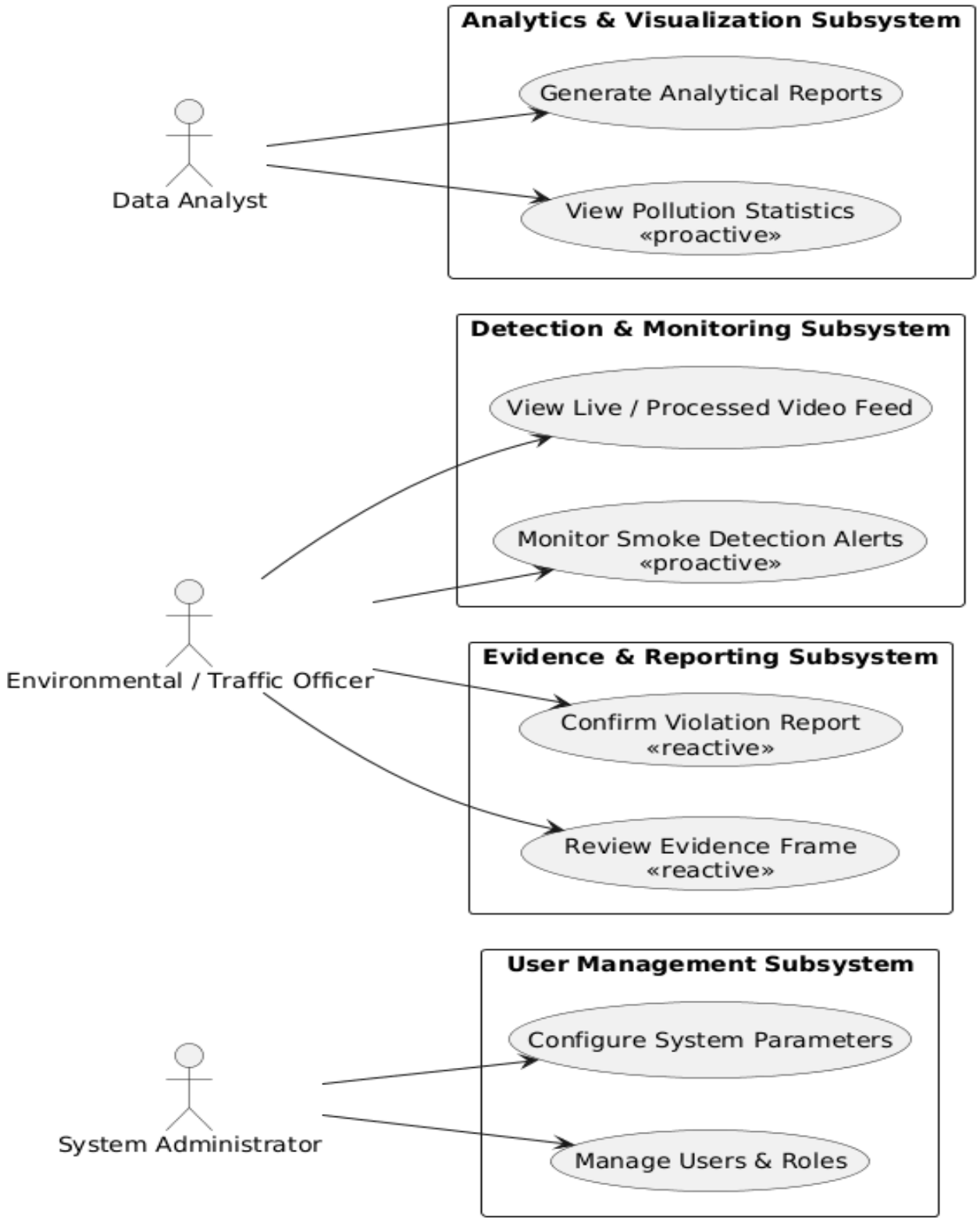
- **Proactive Use Cases**
  - Monitor Smoke Detection Alerts
  - View Pollution Statistics

  These use cases are initiated automatically or periodically by the system based on incoming data or scheduled analysis.

- **Reactive Use Cases**
  - Review Evidence Frame
  - Confirm Violation Report
  - Manage Users & Roles

  These use cases are triggered in response to system outputs or administrative actions.

UML Use Case Diagram (System Level)



Notes on Scope Control

- The use case model intentionally **omits internal AI model operations**, frame-selection logic, and cryptographic processes, as these are internal system responsibilities.
- Third-party systems such as **traffic cameras and storage services** are abstracted and treated as external interfaces.
- This level of modeling ensures **maximum coverage** of system behavior while keeping the diagram suitable for academic review and future expansion.

Domain Model / Entity Model

This section presents the **system fundamentals** using both a **Domain Model** and a **Data Model**. The domain model identifies conceptual entities that participate in system operations, whereas the data model focuses on persistent entities required for system execution and reporting. The separation ensures conceptual clarity while maintaining database normalization and implementation feasibility.

Domain Model

The domain model identifies **candidate entities** using a **noun-hunting technique**, derived from the approved proposal, use cases, and system description. Only entities that directly contribute to system behavior are included.

Noun-Hunting Summary

From problem statements and use cases, the following nouns were identified:

- Vehicle
- Camera
- Smoke Detection
- Evidence Frame
- Violation Record
- User
- Role
- Report

Each noun was evaluated based on **relevance, persistence, and responsibility**. Abstract or purely technical elements (e.g., neural network internals) were intentionally excluded to maintain system-level abstraction.

Domain Entities and Justification

1. Vehicle

Represents a physical vehicle observed in camera footage.
A vehicle may be associated with multiple detection events over time.

2. Camera

Represents a surveillance source providing video input.
A camera acts as a containment entity for detection events.

3. SmokeDetection

Represents an AI-generated event indicating the presence of visible smoke.
This entity links the vehicle, camera, timestamp, and confidence score.

4. EvidenceFrame

Represents a single high-quality image frame extracted from video at the time of detection.
It is functionally dependent on a SmokeDetection event.

5. ViolationRecord

Represents a verified violation after human review.
This entity is created only after officer confirmation.

6. User

Represents an authenticated system user.
Users interact with the system according to assigned roles.

7. Role

Defines access permissions and responsibilities.
Supports Role-Based Access Control (RBAC).

8. Report

Represents analytical or enforcement reports generated from stored data.
Used primarily by analysts and officers.

Candidate entities were identified using noun-hunting technique by analyzing functional requirements and system descriptions provided in Deliverable-01. Only entities with persistent relevance to system operations were retained to avoid unnecessary complexity.

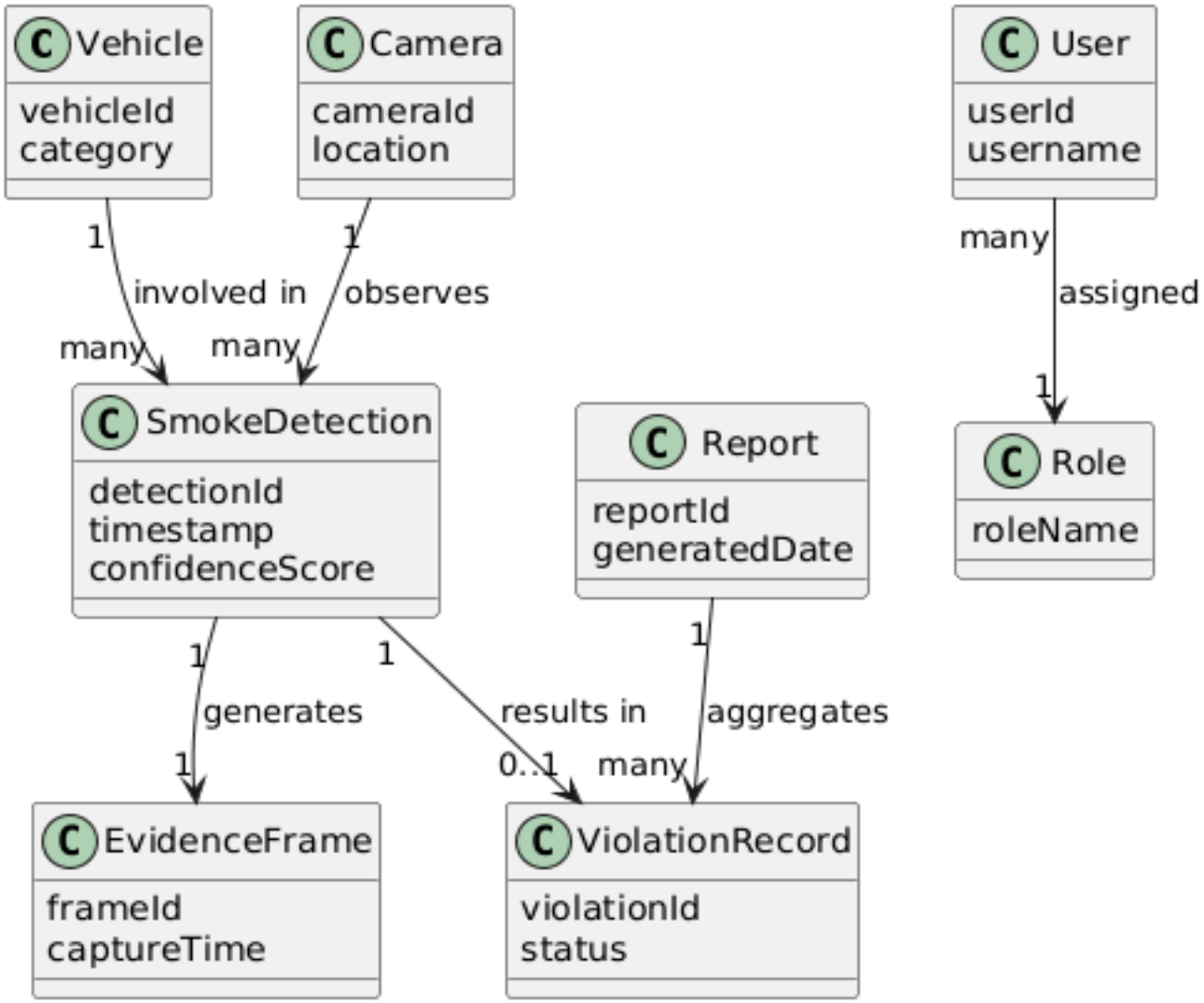
Relationships in Domain Model

- A **Camera** observes multiple **SmokeDetections** (containment)
- A **Vehicle** may be involved in multiple **SmokeDetections**
- A **SmokeDetection** generates exactly one **EvidenceFrame**
- A **SmokeDetection** may result in one **ViolationRecord**
- A **User** is assigned exactly one **Role**
- A **Report** aggregates data from multiple **ViolationRecords**

Functional attributes (e.g., confidenceScore, timestamp) are included only where necessary for understanding system behavior.

Relationships among entities represent real-world interactions and are defined using appropriate associations such as containment and dependency, ensuring conceptual clarity and consistency with system behavior.

UML Domain Model (Class Diagram)



ER Model (Data Model)

The ER model represents **persistent data entities** required for system operation. It is derived from the domain model but refined to ensure **database normalization** and efficient querying.

Normalization

- Redundancy is minimized up to **Third Normal Form (3NF)**.
- Large binary data (images) are isolated into a dedicated entity to reduce table load.

Key Data Entities

- VEHICLE
- VIOLATION
- EVIDENCE
- USER
- RECYCLE_BIN

Binary image data is stored as **BLOB**, referenced through foreign keys.

Object-Relational (O-R) Mapping Technique

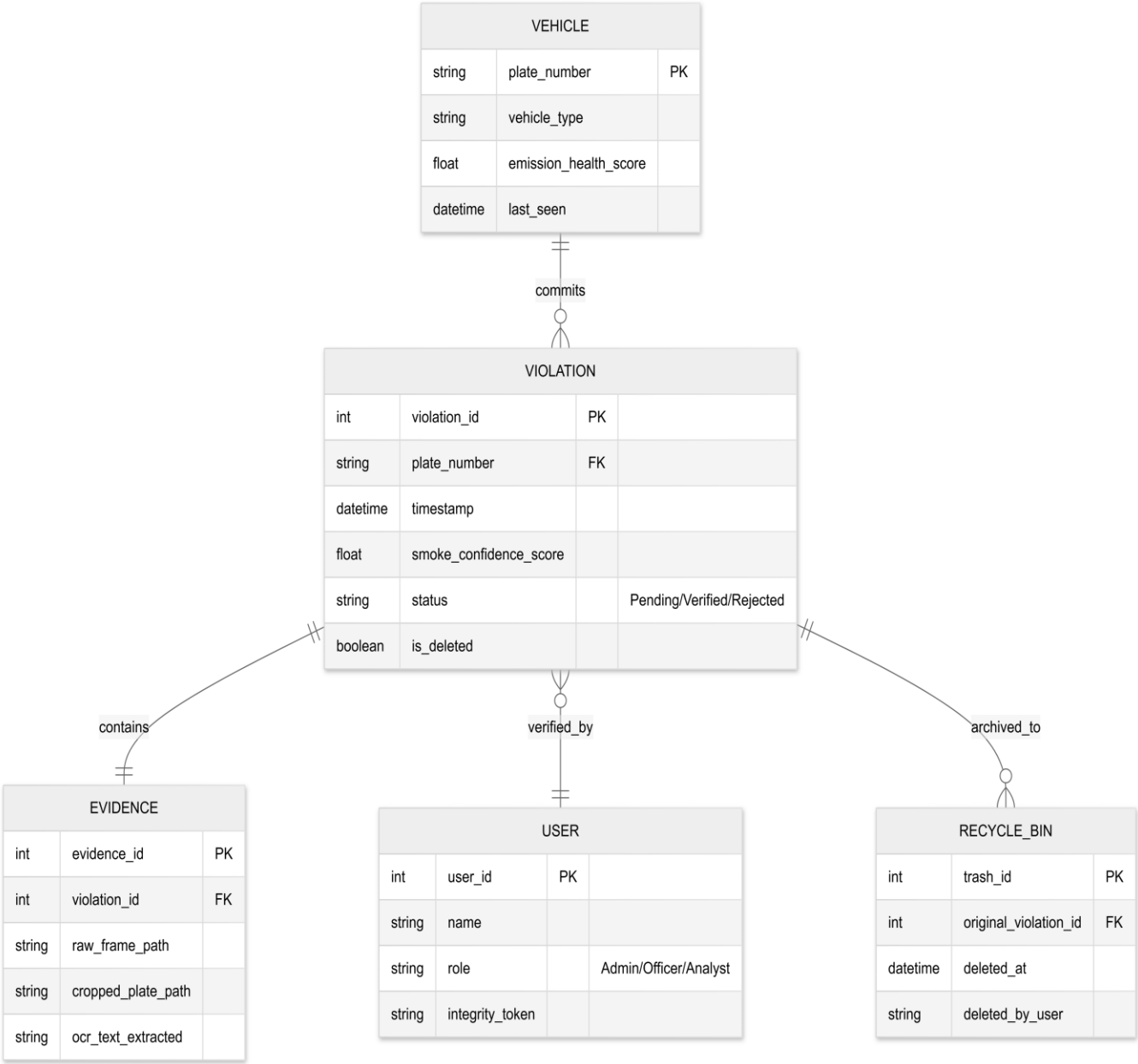
The system follows a **table-per-entity mapping strategy**, where:

- Each domain entity corresponds to a relational table.
- Primary keys uniquely identify records.
- Foreign keys enforce referential integrity.
- BLOB data is stored separately and referenced logically.

This approach ensures compatibility with relational databases and supports future migration to cloud-based storage solutions.

ER Diagram (Normalized)

The ER model is normalized up to at least Third Normal Form (3NF) to minimize redundancy and maintain data integrity. A suitable Object-Relational Mapping (ORM) approach is considered to map domain objects to relational entities, ensuring consistency between object-oriented design and database schema.



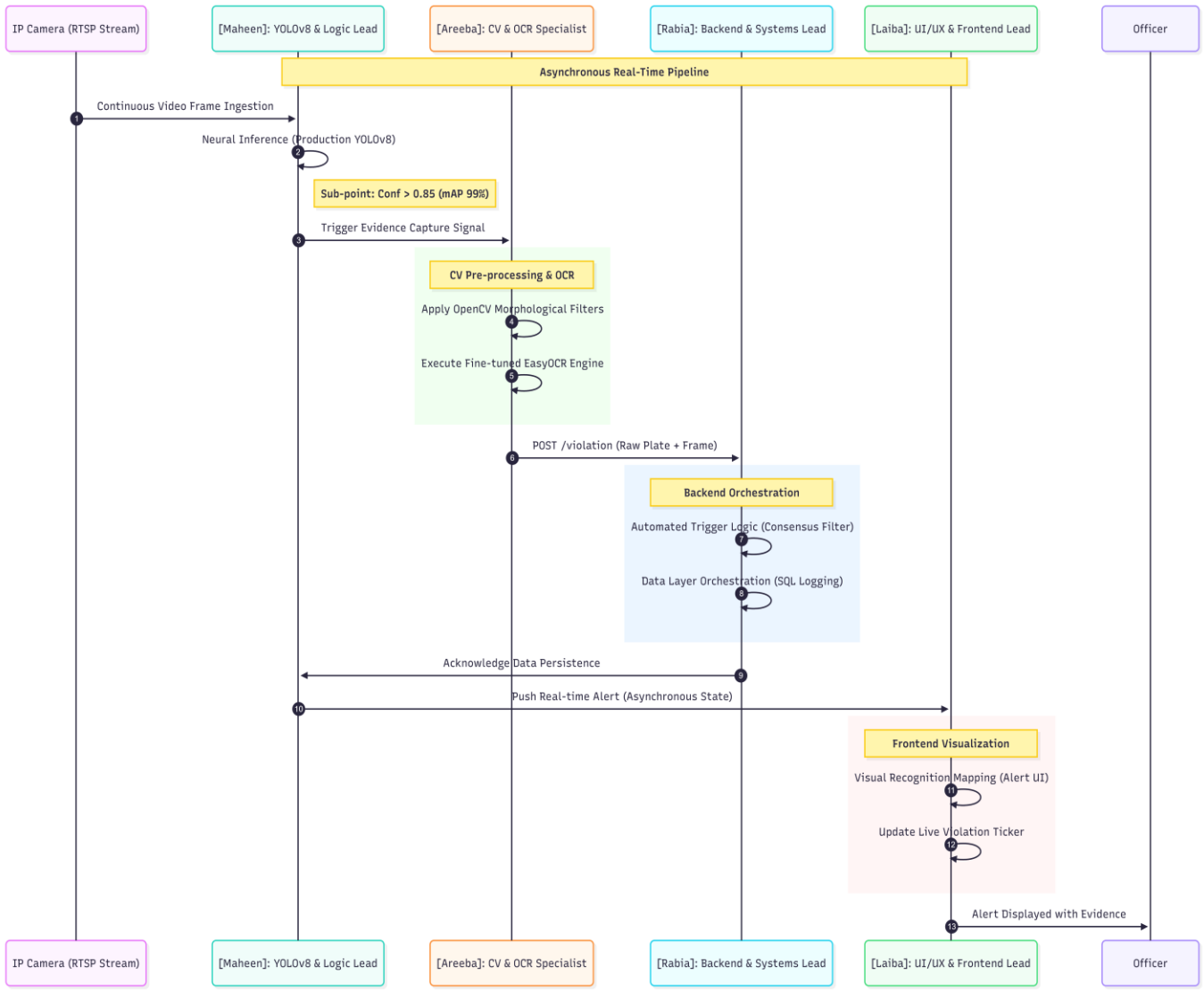
Description

1. **Normalization & Scalability:** The ER Model is designed in **3rd Normal Form (3NF)** to ensure that even with **50,000+ records**, query latency remains minimal. The separation of `EVIDENCE` from the `VIOLATION` table ensures that heavy image paths don't slow down basic statistical queries.
2. **The Integrity Shield Logic:** In the `USER` table, we maintain an `integrity_token`. Any verification action on a violation is cryptographically linked to the officer's ID, ensuring that evidence cannot be tampered with—fulfilling our **"Integrity Shield"** promise.

3. **<u>Soft-Delete Architecture (Recycle Bin):</u>** Aligned with our **D1 scope**, the `RECYCLE_BIN` table handles "Soft-Deletes." When a record is "deleted," it isn't removed from the disk; instead, the `is_deleted` flag is toggled, and a mirror entry is created in the Recycle Bin for a 30-day recovery window.
4. **<u>Emission Health Scoring:</u>** The `VEHICLE` table tracks an `emission_health_score`. This is a cumulative metric that identifies "Super-Emitters" by analyzing how many times a specific plate has triggered the YOLOv8 smoke detection engine.

System Behavioral Model (Sequence Diagram)

The following sequence diagram illustrates the dynamic interaction between the AI inference engines, the backend orchestration layer, and the real-time user interface.



The Technical Narrative

As illustrated, the system maintains a continuous frame-ingestion loop. Upon a high-confidence smoke detection by the **YOLOv8 module** , a trigger is sent to the **OCR pipeline** for plate extraction. The **FastAPI backend** then performs temporal validation before broadcasting the event via WebSockets to the **React Dashboard** , ensuring sub-second notification latency.

Page Design

For this web-based system, structured page layout design is used to demonstrate workflow clarity, role-based interaction, and operational efficiency. Wireframe-oriented mock designs are adopted to focus on functionality, hierarchy, and user flow rather than aesthetic styling.

The page layouts follow a **role-driven interface model**, ensuring strict enforcement of Role-Based Access Control (RBAC). Each user category interacts only with the features relevant to their responsibilities. The design prioritizes clarity, minimal cognitive load, and rapid decision-making in enforcement scenarios.

The number of pages is intentionally limited yet sufficient to illustrate the complete lifecycle of detection, verification, reporting, and administration.

Page 1: Login Page

Purpose

Authenticate users and enforce secure role-based access.

Design Logic

The login page acts as the security gateway of the system. It ensures that only authorized personnel access sensitive environmental enforcement data.

Key Interface Elements

- Username input field
- Password input field
- Secure login button
- Error/validation message placeholder
- System branding header

Workflow

User → Credential Verification → Role Identification → Redirect to Authorized Dashboard

Security measures such as encrypted credential transmission and controlled session handling are assumed at backend level.

Page 2: Role-Based System Dashboard

Purpose

Provide a centralized operational overview tailored to user roles.

Design Logic

The dashboard uses a structured card-based layout. Information is organized in priority order to support quick situational awareness.

Role Variations

Administrator Dashboard

- System health indicators
- User management shortcut
- Recent activity logs

Officer Dashboard

- Live alert panel (near real-time updates)
- Pending violation confirmations
- Recently reviewed cases

Analyst Dashboard

- Statistical summary cards
- Trend graphs
- Report generation shortcut

Key Interface Elements

- Left-side navigation panel
- Summary metric cards
- Notification/alert area
- Quick action buttons

UX Enhancement

Alerts are visually distinguished using color indicators (e.g., high-confidence detections highlighted distinctly) to support rapid attention and prioritization.

Page 3: Smoke Detection & Alerts Page

Purpose

Display detected smoke events in structured chronological format.

Design Logic

This page represents the operational core for enforcement officers. Alerts are listed dynamically and updated through asynchronous communication mechanisms (WebSocket-based event updates in real-time deployment).

Key Interface Elements

- Scrollable alert list
- Timestamp
- Camera/location reference
- Detection confidence indicator
- "View Evidence" action button

Workflow

Detection Event → Alert Generated → Officer Reviews Alert

High-confidence detections may be visually emphasized to assist decision-making under time constraints.

Page 4: Evidence Review Page

Purpose

Enable human verification of AI-detected violations.

Design Logic

This page ensures that final enforcement decisions remain under human supervision, preserving accountability and reducing false positives.

Key Interface Elements

- Evidence image (captured frame snapshot)
- Detection metadata:
    - Timestamp
    - Camera ID
    - Confidence score
- Confirm Violation button
- Reject Detection button

Workflow

Alert → Evidence Review → Officer Decision → Violation Record Created (if confirmed)

This page directly supports the "Integrity Shield" logic described in the ER Model, ensuring traceable confirmation actions.

Page 5: Reports & Analytics Page

Purpose

Support analytical insight and environmental monitoring.

Design Logic

The page presents aggregated data views for trend analysis and policy-level decision support. Data visualization components are structured to maintain readability even with large datasets.

Key Interface Elements

- Date and location filters
- Emission trend charts
- Repeat offender identification
- Report export option

The interface is designed to scale with growing historical records (50,000+ entries) without affecting readability.

Page 6: User Management Page (Administrator Only)

Purpose

Manage users, roles, and system access privileges.

Design Logic

This interface enforces controlled access modification. Only administrators can create, modify, deactivate, or assign roles to users.

Key Interface Elements

- User listing table
- Role assignment dropdown
- Account activation/deactivation controls
- Search/filter options

All administrative changes are logged for accountability and traceability.

Functional Alignment

Each page directly corresponds to one or more validated use cases described in the Use Case Model:

- Login Page → Authenticate User
- Dashboard → Monitor Alerts / View Statistics
- Detection & Alerts Page → Monitor Smoke Detection
- Evidence Review Page → Confirm Violation
- Reports Page → Generate Analytical Reports
- User Management Page → Manage Users & Roles

This ensures traceability between requirements, UML diagrams, and interface design.

Non-Functional Design Considerations

The page design also supports key quality attributes of the system:

- **Performance:** Interfaces are structured to support asynchronous updates (WebSocket-based alerts) without requiring manual refresh.
- **Scalability:** Layout components (tables, charts, filters) are designed to handle large datasets (50,000+ records).
- **Security:** Authentication and role-based redirection prevent unauthorized access to enforcement data.
- **Usability:** Information hierarchy prioritizes critical alerts over secondary statistics.
- **Maintainability:** Modular UI sections allow independent enhancement without disrupting core workflow.

Layout Structure & Orientation

The system follows a responsive web layout strategy:

- Left-aligned navigation panel for stable orientation
- Centralized content area for primary interaction
- Top notification bar for real-time alerts
- Card-based dashboard elements for structured information grouping

The orientation is optimized for desktop-first deployment (monitor-based command centers), while remaining adaptable for tablet-scale screens in future expansion.

UX Design Principles

The interface design is guided by:

- Minimal cognitive load
- Clear action visibility (Confirm / Reject decisions)
- Visual prioritization of high-confidence detections
- Consistent navigation structure across all roles
- Logical progression from detection → verification → reporting

The focus remains on operational efficiency and decision accuracy rather than decorative styling.

Design Scope Control

Color schemes, typography specifics, and branding themes are intentionally not over-specified at this stage, as ADD2 focuses on structural and behavioral design rather than final UI theming. Visual refinements will be addressed during implementation phase.

CASE Tools

This section lists the **Computer-Aided Software Engineering (CASE) tools** selected for the design and development of the proposed system. The chosen tools support system modeling, design documentation, database design, and interface prototyping. Selection is based on **compatibility, academic suitability, ease of use, and future extensibility**.

The tools are widely adopted, platform-independent, and compatible with standard development environments, ensuring smooth transition from design to implementation.

CASE Tools Selection Table

Sr. #	Category	CASE Tool	Appropriate Reasoning for Choice
01	Architectural Modeling	Draw.io & PlantUML	Utilized for designing high-level **Use Case** and **ER Models**. PlantUML specifically enabled "Diagrams-as-Code" for precise logic modeling.
02	AI Training Environment	Google Colab	Leveraged for cloud-based **GPU acceleration** (T4) during the YOLOv8 fine-tuning phase on the dataset.
03	Data Ops & Annotation	CVAT & Roboflow	**CVAT** was used for initial manual labeling, while **Roboflow** managed dataset health checks, augmentations, and format conversions.
04	Workflow Automation	n8n	**(Advanced Integration)** Used to automate the alert pipeline. It triggers external notifications (Emails/Alerts) to regulatory bodies once a violation is committed to the database.
05	Agile Project Management	Jira (Atlassian)	Used for managing **2-week Sprints** and tracking backlogs. Jira's professional-grade monitoring ensures the team meets strict university timelines.
06	API Testing	Postman	Essential for stress-testing **FastAPI** endpoints and validating the JSON handshake between the AI core and the administrative dashboard.

07	UI/UX Design	Figma	Used for high-fidelity prototyping of the real-time Command-and-Control interface before implementing in **React 18**.
08	Version Control	GitHub	Centralized repository for code management, ensuring the integrity of the **"Integrity Shield"** and **"Recycle Bin"** logic across all team members.
09	Cloud Logic & Hosting	Vercel / Hostinger	Used for the continuous deployment (CD) of the React 18 dashboard. It demonstrates the system's availability for remote monitoring by regulatory authorities via live URLs.
10	Real-time Communication	Socket.io / WebSockets	(Programmatic CASE) Essential for the "Live Alert" feature. It enables the FastAPI backend to push detection events to the frontend in under 100ms, eliminating the need for page polling.
11	Database Management (GUI)	MongoDB Compass / TablePlus	Used for visual database schema design and auditing. Crucial for managing the 50,000+ historical records and verifying the "Integrity Shield" log entries.
12	Performance Benchmarking	TensorBoard	Utilized during the YOLOv8 training phase to visualize loss curves, mAP growth, and precision-recall trade-offs. It provides the "Scientific Proof" of the model's 99% accuracy.
13	Network Tunneling	Ngrok	Used during the development phase to expose the local FastAPI server to external IP cameras, allowing for real-world stress testing of the smoke detection engine.

Management & Communication CASE Tools

Sr. #	Category	CASE Tool	Appropriate Reasoning for Choice
01	Progress Tracking	Microsoft Excel (Work Logs)	Utilized for maintaining a detailed **FYP Work Log**. It tracks individual task contributions, deadlines, and milestone completion, ensuring transparency in team performance.
02	Synchronous Collaboration	Google Meet	Used for conducting **Sprint Reviews** and virtual technical syncs. It facilitated screen-sharing sessions for live code debugging and model training reviews.
03	Asynchronous Sync	WhatsApp	Primary channel for instant communication, quick technical updates, and maintaining a real-time link between the AI research and frontend development phases.

CASE Tools Summary

The selected CASE tools collectively support:

- UML-based system modeling
- Normalized data modeling
- Page workflow visualization
- Academic documentation standards
- Future scalability and maintainability

These tools ensure design consistency across Deliverables and facilitate a smooth transition into the implementation and testing phases of the project.

Appendix

A. Overview of CASE (Computer-Aided Software Engineering) Tools

CASE tools are software utilities that assist in various phases of the Software Development Life Cycle (SDLC), including requirement analysis, system design, documentation, testing, deployment, and project management.

In this project, CASE tools are selected based on:

- Compatibility with system architecture
- Academic suitability
- Professional industry relevance
- Cost-effectiveness (preference for open-source tools)
- Ease of integration with development environments

The selection ensures consistency across design artifacts, implementation modules, and documentation deliverables.

B. CASE Tools for System Modeling

System modeling is a critical part of Deliverable-02 (ADD2), where architectural clarity and structured representation are required.

PlantUML

PlantUML is used as the primary UML modeling tool. It supports diagram generation through textual scripting, enabling version control, reproducibility, and precise logical structuring of diagrams. It is used for:

- Use Case Diagrams
- Domain Model (Class Diagram)
- Sequence Diagrams
- ER Diagrams

Its compatibility with IDE environments and Git-based workflows ensures maintainability of design artifacts.

draw.io

draw.io is used as a visual diagram refinement tool. It supports exporting high-resolution diagrams suitable for academic submission and presentation. It complements PlantUML for layout adjustments and final formatting.

C. CASE Tools for AI Development & Validation

The AI component of the system requires model training, dataset preparation, and performance validation.

Google Colab

Google Colab provides cloud-based GPU acceleration for YOLOv8 model training and experimentation. It supports collaborative notebooks and reproducible research workflows.

CVAT

CVAT (Computer Vision Annotation Tool) is used for manual dataset labeling of smoke instances in vehicle footage.

Roboflow

Roboflow is utilized for dataset management, preprocessing, augmentation, and format conversion compatible with YOLOv8 training requirements.

TensorBoard

TensorBoard is used for monitoring model performance metrics such as:

- Loss curves
- Precision
- Recall
- Mean Average Precision (mAP)

This ensures scientific validation of the detection model.

D. CASE Tools for Backend & API Testing

Postman

Postman is used for API endpoint validation and stress testing of FastAPI services. It verifies:

- Request-response consistency
- JSON data integrity
- Authentication token handling

This tool ensures reliable communication between AI inference pipeline and frontend dashboard.

E. CASE Tools for Real-Time Communication & Deployment

WebSockets / Socket.io

Used to implement real-time alert broadcasting from backend to dashboard without polling. This supports the system's asynchronous architecture.

Vercel / Hostinger

Used for hosting and deployment of the frontend dashboard. Continuous deployment ensures availability for remote monitoring.

Ngrok

Used during development for secure exposure of local backend services to external traffic cameras for real-world testing scenarios.

F. CASE Tools for Repository & Version Control

GitHub

GitHub is used for centralized version control and collaborative development. It supports:

- Branch management
- Commit tracking
- Issue management
- Artifact traceability

It ensures controlled evolution of the project and protects system integrity.

G. CASE Tools for Project Management & Communication

Jira

Used for sprint planning and backlog tracking under an Agile workflow. It ensures task transparency and milestone monitoring.

Microsoft Excel (Work Logs)

Maintains structured documentation of weekly progress, individual contributions, and deliverable tracking.

Google Meet

Used for technical discussions, sprint reviews, and collaborative debugging sessions.

WhatsApp

Used for quick coordination and real-time team communication during development cycles.

Appendix Summary

The selected CASE tools collectively support:

- Structured UML and ER modeling
- AI model training and validation
- API development and testing
- Real-time system architecture
- Version-controlled collaborative development
- Agile project management

The tools are aligned with academic standards while maintaining professional relevance. They ensure smooth transition from design (ADD2) to implementation and final deployment phases.